F29AI Edinburgh CW 1

Group 49

Link to video: https://youtu.be/HKO1uDCShQs

No GenAi was used in the making of this report or in any of the code created by us

Rory McMiken – H00465360

Louis Cole – H00475217

Part 1A:

The variables in this CSP are the 81 cells of the sudoku grid. The domains are subsets of {1..9}, which are stored as 9-bit integers. The constraints are that two cells can conflict when they have the same number and are in the same row, column or box, there are also constraints that each row must contain each of the digits from 1-9 exactly once and the same goes for the columns and squares, they must also contain each of the digits from 1-9 exactly once.

Brute force fills all the empty cells with digits 1-9 and then tests the constraints to try multiple solutions and discount possible solutions early. This method would have a worst case time constraint of O(9^{81}) as if every cell was empty then there would be 9 possible values for each cell. This would change if there was a variation in the empty number of cells as sudoku usually starts with some cells filled therefore the worst case in a scenario with some cells filled would be O(9^{E}) where E is the number of empty cells.

Using a backtracking search would assign the values to the individual cells one at a time while checking against the constraints after each assignment. If a conflict arises when assigning one of the cells the algorithm backtracks to an earlier cell to try another value. Similarly to brute force backtracking also has a worst case time complexity of O(9^{E}) as in the worst case it could still explore an exponential amount of assignments of digits before finding the solution.

Using backtracking we are able to use pruning, which allows us to stop following a path once its clear that a solution isn't possible from that path. This allows us to see a problem before it arises and there are many useful effects of pruning, such as that many branches are abandoned early before they're fully explored this also means that the algorithm avoids completing grids that it knows wont be valid, this then leads to a smaller search tree as the number of assignments that are examined would be lower, even though the theoretical search space would be exponential.

Part 1B code:

Our first task was to create a Sudoku solver using Python. It reads an incomplete 9-by-9 Sudoku puzzle from a text file, checks that it is valid, and attempts to solve it. The program uses backtracking, a CSP technique based on trial and error. It tries placing numbers into empty cells and, if a choice leads to a problem, goes back and tries a different number.

At the beginning, we import the path module, which allows us to access the file with the sudoku puzzle in it. We define two main variables: size, set to 9 for the number of rows and columns, and box_size, set to 3 for the size of each smaller box.

The first function, read_sudoku, reads the puzzle from a text file and converts it into a two-dimensional list. It removes unnecessary spaces and formatting characters, and treats dots and zeros as empty cells. It also checks that the puzzle contains exactly nine rows, with nine cells in each row. If the format is incorrect, the program produces an error.

The is_valid_grid function checks whether the numbers already in the puzzle follow the Sudoku rules. It goes through each cell, skipping empty ones, and temporarily removes each existing number so it can check whether that number is valid in its original position. It then restores the number.

To perform these checks, it calls is_valid_value. This function checks three rules: a number must not appear more than once in the same row, column, or 3-by-3 box. It returns True if the number is valid and False if it breaks any of these rules.

The solve_sudoku function is the main part of the program. It uses **recursion and** backtracking to solve the puzzle.It searches the grid from left to right, row by row, until it finds an empty cell. It then tries numbers from 1 to 9, using is_valid_value to check each one.When a valid number is found, it places it in the cell and calls the function again to solve the next empty cell. This is called recursion. If a choice eventually leads to a cell where no number works, the function returns False. The previous number is then removed, and the program tries the next possibility. This is backtracking. If there are no empty cells left, the function returns True, meaning the puzzle has been solved.

The print_sudoku function displays the puzzle in a readable format. It prints the numbers row by row and adds horizontal and vertical lines to separate the 3-by-3 boxes.

The main function controls the program. It checks that a text file has been provided as a command-line argument, reads the puzzle, and handles any errors.It then displays the original puzzle and attempts to solve it. If successful, it prints the completed Sudoku. Otherwise, it displays a message saying that no solution was found.the condition at the bottom of the code calls main when the Python file is run directly.

Part 1B: Theoretical A* Analysis of the Sudoku Solver

A* is an informed search algorithm that selects states using:

[
f(n)=g(n)+h(n)
]

State: The current 9×9 Sudoku grid.

Initial state: The incomplete puzzle loaded from the text file.

Actions: Place a valid number from 1 to 9 in an empty cell.

Path cost, (g(n)): Number of assignments made so far, assuming each costs 1.

Goal state: A completed grid satisfying all Sudoku rules.

Heuristic, (h(n)): Number of empty cells remaining.

This heuristic is admissible because every empty cell must be filled. However, it is weak as each assignment increases (g(n)) by one and decreases (h(n)) by one, leaving (f(n)) unchanged. Also, A* cannot effectively distinguish promising states from unpromising ones.

Comparing A* to backtracking which was used in our code we get:

Feature	**Current backtracking**	**A* search**
Search strategy	Explores a branch until it fails, then backtracks.	Expands the state with the lowest (g(n)+h(n)).
Time complexity	Exponential in the worst case.	Exponential in the worst case.
Memory usage	Relatively low; stores the current search path.	Potentially high; stores many alternative states.
Main advantage	Simple and effective for finding a valid solution.	Can find minimum-cost solutions with a suitable heuristic.
Main disadvantage	Poor cell ordering can cause many backtracks.	Requires a useful heuristic and additional memory.

A* is less suitable here because Sudoku requires a valid solution, not necessarily the cheapest one. With unit-cost assignments, every complete solution requires the same number of assignments.

Backtracking is a suitable approach for this Sudoku solver because it is straightforward and uses relatively little memory. A* is possible, but the simple empty-cell heuristic provides little guidance and its memory requirements can be much higher.

Part 2:

For the PDDL section I will first describe the important design decision with object types and important predicates before moving on to actions. The planner we used in this project was LAMA as it was one of the options suggested in the brief and from researching the 2 this one seemed better for short tasks which ours.

Types

When defining types for the missions I started with 4 sub-types from object: Rover, Lander, Collectable and waypoint. Then as sub types of collectable, I had sample and data then finally photo and scan as subtypes of data. The reason that each type of collectable isn't a sub-type of object is so that predicates and actions can be made more general, for example to define the location of a scan, photo or sample I can use one predicate as the location aspect of each collectable type functions the same way. By also having photo and scan as sub-types of data we can also make more general predicates for both of them as they both function identically which make the code simpler over all

Predicates

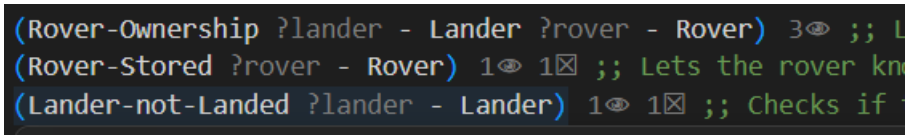


Figure 1: Lander Predicates

The first important predicates we will look at shown in Figure 1 notably with Lander-not-Landed. We had the option to set the landers position manually or let the planner choose so we did the latter option it was interesting to see that for all mission the planner chose to land on the waypoint where a sample needs to be collected though this makes sense as it save the rover having to drive to the sample and back.

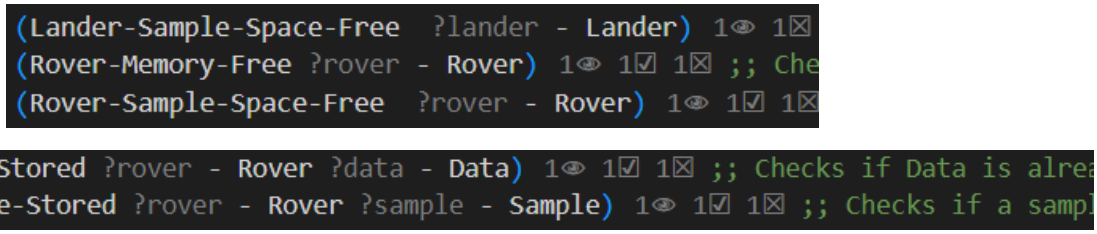


Figure 2: Storage Predicates

This section of predicates covers the storage restriction of the rover and the lander. In the mission description we are told that the rovers can only hold 1 piece of data and 1 sample at a time and that a lander can only hold 1 sample. Since they can only hold a single piece of data we can just check if data is stored as a sort of true or false. Since the strips data type can only check if something is true when defining an action, we also need to have another predicate to check the rovers is storing a sample or some data.

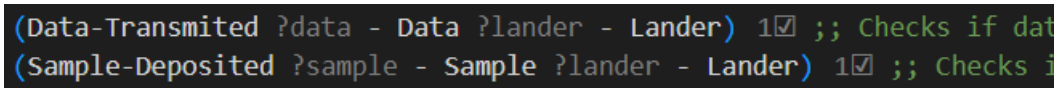


Figure 3: Goal Predicates

We then have 2 predicates that checks when a collectable has been either transmitted or deposited in a lander. These are the predicate that act as the goal for the planner as the aim of the mission is to collect all the data and samples so once we can flag all the Sample-Deposited predicates true we know that all the samples have been collected.

Actions

```
(:action Capture-Data   ;; Lets the rover capture data either a sca
    :parameters (?rover - Rover ?data - Data ?location - Waypoint)
    :precondition (and (Collectable-Location ?data ?location)   ;;
                       (Rover-Location ?rover ?location)    ;; Chec
                       (Rover-Memory-Free ?rover))  ;; Checks if th
    :effect (and (not (Rover-Memory-Free ?rover))
                 (not (Collectable-Location ?data ?location))
                 (Rover-Data-Stored ?rover ?data))
)
```

Figure 4: Action to capture data

For the capture data action, the effects are interesting to look at as we can see the Rover-Memory-Free and Rover-Data-Stored staying opposite as discussed earlier letting different action use the type they need. We also remove the data's location in the effects too, this is done so that the same piece of data can't be collected multiple times which could potentially cause problems.

```
(:action Transmit-Data  ;; Lets the rover send collected data b
    :parameters (?rover - Rover ?lander - Lander ?data - Data)
    :precondition (and (Rover-Data-Stored ?rover ?data) ;; Chec
                       (Rover-Ownership ?lander ?rover)) ;; Che
    :effect (and (not (Rover-Data-Stored ?rover ?data))
                 (Data-Transmited ?data ?lander)
                 (Rover-Memory-Free ?rover))
)
```

Figure 5: Transmit Data action

Finally, we have the action to transmit data to the rover. Since data can be transmitted wirelessly, we don't need to check the lander and rovers' location we only check that the rover has data to transmit and that its transmitting to the correct lander. The action for depositing a sample works similarly but it also considers the location of the rover and lander as they need to be at the same waypoint to deposit the sample.

We also have other action that are used for moving the rover and deploying the rover however they are fairly basic and work relatively intuitively so won't be discussed here.

Astronaut

In part C of the PDDL task we needed to add astronauts to the landers which need to be in specific rooms to do certain tasks. To do this we need to 4 new types the first 2 are astronaut and room which will be sub-types of object and then control-room and docking-bay which will be sub-types of room.

```
(Astronaut-Lander ?astronaut - Astronaut ?lander - Lander) 4👁  ;; All
(Astronaut-Room ?astronaut - Astronaut ?room - Room) 4👁 1☑ 1☒    ;;
(Lander-Room-Connection ?lander - Lander ?room - Room) 5👁  ;; Links a
```

Figure 6: Astronaut Predicates

We also add 3 new predicates for the astronauts and rooms, the Astronaut-Lander and Lander-Room-Connection predicates are used to link either an astronaut or a room to a specific lander, so we don't accidentally have astronauts travelling between landers.

```
(:action Transmit-Data  ;; Lets the rover send collected data back to its lander wirelessly
    :parameters (?rover - Rover ?lander - Lander ?data - Data ?astronaut - Astronaut ?room - Control-Room)
    :precondition (and (Rover-Data-Stored ?rover ?data) ;; Checks that the rover is actually carrying data
                       (Rover-Ownership ?lander ?rover) ;; Checks if the lander we are trying to send the
                       (Astronaut-Lander ?astronaut ?lander)    ;; Checks that the astronaut in the contro
                       (Astronaut-Room ?astronaut ?room)    ;; Checks that the astronaut is in the control
                       (Lander-Room-Connection ?lander ?room))  ;; Checks that the control room that is be
    :effect (and (not (Rover-Data-Stored ?rover ?data))
                 (Data-Transmited ?data ?lander)
                 (Rover-Memory-Free ?rover))
)
```

Figure 7: Transmit Data action for mission 3

Here we see the transmit data action now updated to include the astronauts. The way we are checking that the astronaut is in the correct room is by setting the type of the room parameter to be specifically control-room and since it is a subtype of room the Astronaut-Room predicate still works but now only if the astronaut is in the control room. This same idea of setting the specific type to the room required is also used for the other actions that require astronauts.

With the astronauts we also had to add another action for moving between rooms since we used a more general definition of room with 2 sub-types we can make this 1 action however there is 1 problem that was found with this is that can travel between the same room within the same lander which is harmless but could waste turns if the planner decided to take that action.

With the astronaut changes the number of moves required to complete the mission increased from 21 to 25 which were just the movements of the astronauts to the correct room for the task

Testing

To test the domain and problem files were producing viable plans we primarily used paper to check the plan with a drawing of the map and collectable locations so we could quickly trace the path out to check it was viable. While this worked will for the first mission as it was quite short, but it wasn't a good option for the longer missions though we did still continue to use it.

This method did help us find some bug in the code for example we entered a connection wrong instead of connecting 5 to 3 we connected 4 to 3 which was allowing the rover to travel a path it shouldn't have. We also had a typo where bob started in control-room-1 not control-room-2 so the mission wasn't possible, this bug was found just by checking the new code added in part 3 so see where it was going wrong.